Final Modeling and Simulation Project

A Guide to Developing, Testing, and Understanding a Computational Model

PHYS 441

The central principle: A simple model that you understand deeply, verify carefully, and investigate thoughtfully is preferable to an ambitious model that you treat as a black box.

1 Purpose of the project

The final project is an opportunity to take responsibility for a complete modeling and simulation problem. You will choose a system, formulate a focused question, implement a model, test the implementation, investigate its behavior, and explain what the results do and do not mean.

Your project does *not* need to be brand new, publishable, or earth-shattering. You may begin with a model from this course, another course, your research, a textbook, or a published example. Originality can be valuable, but it is not the primary goal. The goal is intellectual ownership.

By the end of the project, you should be able to say:

> I understand what this model assumes, how it is implemented, how I tested it, where I encountered difficulties, what its results mean, and what its limitations are.

This is an **individual project**. Discussion with classmates is welcome, but every student must formulate, implement, investigate, and explain an individual project.

2 Learning objectives

The project will give you practice in the full modeling workflow. You will:

- translate a broad topic into a specific, answerable question;
- identify state variables, parameters, governing equations, and modeling assumptions;
- convert a mathematical or rule-based model into a working computation;
- distinguish verification from scientific interpretation;
- design checks that can expose incorrect or unreliable results;
- conduct a controlled computational investigation rather than a single demonstration run;
- diagnose and document difficulties encountered during implementation;
- communicate results, uncertainty, and limitations clearly; and
- produce code and documentation that another person can understand and reproduce.

3 Choosing an appropriate project

Begin with a question, not merely a topic. A topic such as "epidemics," "orbits," or "random walks" is too broad to guide an investigation. A useful project question identifies a relationship, comparison, prediction, or behavior that the model can examine.

Too broad	**Focused enough to investigate**
Simulate an epidemic.	How do recovery rate and initial infection fraction affect peak infection and outbreak duration in an SIR model?
Model an orbit.	How does numerical energy drift depend on time step for Euler, midpoint, and fourth-order Runge–Kutta orbit integrators?
Study random walks.	How does the mean first-passage time depend on boundary distance for unbiased and biased one-dimensional random walks?
Simulate diffusion.	How do grid spacing and time step affect the stability and accuracy of an explicit finite-difference diffusion model?

An appropriate project has all of the following characteristics:

- The question is narrow enough to address during the remaining course time.
- The model has identifiable assumptions, variables, and rules or equations.
- A small but complete version can be implemented before extensions are attempted.
- At least one meaningful verification check is possible.
- The model supports a controlled comparison or parameter study.
- The expected output is interpretable, not merely visually impressive.

If you already have a suitable problem from research or another course, you may use it. If you prefer to extend a model from PHYS 441, that is also an excellent choice. If you are unsure what to choose, ask for an instructor-suggested project. These routes are equally legitimate and will be evaluated with the same criteria.

4 Required elements of every project

4.1 A specific question

State the question your simulation will investigate. Your question should tell the reader what will be varied, compared, estimated, or explained. A project may evolve, but it should always have a question that determines what evidence is relevant.

4.2 A clearly described model

Explain the model independently of the code. Identify:

- the physical, biological, social, mathematical, or engineered system;
- the variables that define its state;
- the parameters and their meaning;
- the equations, transition rules, or update rules;
- the initial and boundary conditions, when applicable;
- the assumptions that make the model manageable; and
- important real effects that the model omits.

A diagram, flowchart, table, or pseudocode description may be helpful. The model description must be understandable without reading the source code.

4.3 An implementation you understand

You are not required to write every numerical operation from scratch. You may use standard scientific libraries and may consult existing implementations. However, you must understand every important part of the computation and clearly identify any code or method adapted from another source.

Using a library does not remove the need to explain the model. For example, if you use an ODE solver, you must still explain the equations, initial conditions, integration interval, important solver settings, and how the returned solution is interpreted. Existing software may support your understanding; it may not replace it.

4.4 Verification

Verification asks:

> **Did I implement the model that I intended to implement?**

Every project must include at least one concrete verification test. Possible tests include:

- comparison with a known analytic solution;
- a hand calculation for the first few update steps;
- a limiting case whose behavior is known;
- conservation of energy, mass, probability, or another invariant;
- convergence under smaller time steps or finer spatial grids;
- comparison with an independently written simpler implementation;
- unit tests for important computational functions; or
- a benchmark value from a reliable source.

"The plot looks reasonable" is not, by itself, verification. State the expected result before presenting the test, show the evidence, and explain whether the test passed.

4.5 A controlled computational investigation

A single simulation run is not a complete investigation. After the baseline model works, study its behavior through one or more controlled comparisons:

- vary one or two physically or mathematically meaningful parameters;
- compare multiple initial conditions;
- compare numerical methods, resolutions, or step sizes;
- repeat a stochastic simulation with multiple random seeds;
- compare alternative model assumptions; or
- perform a sensitivity or uncertainty study.

For at least one comparison, record a prediction before running the calculation. Then compare the actual outcome with the prediction. Unexpected behavior is not automatically a failure; it is often the beginning of the most interesting part of the investigation.

4.6 A difficulty and diagnosis record

Real computational work rarely proceeds perfectly from start to finish. Every project must document at least one substantive difficulty. Examples include a bug, instability, excessive run time, unexpected parameter sensitivity, inconsistent units, a result that looked plausible but was wrong, or a model assumption that failed.

For the selected difficulty, explain:

1. What did you observe?
2. What possible causes did you consider?
3. What evidence helped identify the cause?
4. What did you change, if anything?
5. What did you learn from the process?

You are not being graded on having a trouble-free project. You are being graded on responding to difficulties thoughtfully and using evidence.

4.7 Interpretation and limitations

Separate what the program produced from what you infer. Explain the important patterns in the output and connect them to the original project question. Identify at least two limitations, such as simplifying assumptions, uncertain parameters, finite sampling, numerical error, restricted dimensionality, missing physics, or a narrow validation range. Explain how those limitations constrain the conclusions.

5 Scope: core, investigation, and extension

Plan the project in three layers:

Core model – required
The smallest scientifically complete implementation. It should run, produce interpretable output, and support at least one verification test.

Investigation – required
A controlled study that uses the working model to address the project question.

Extension – optional
An additional feature attempted only after the core and investigation work. Examples include another dimension, a more realistic force law, a second numerical method, improved performance, or comparison with data.

Define the core conservatively. A completed and carefully investigated core is more valuable than an unfinished collection of ambitious features. Extensions do not compensate for missing verification or unclear interpretation.

6 Project schedule and milestones

Project work begins during the week of November 9 and continues through final exam week. The checkpoints below are designed to reveal problems early and to protect time for verification, investigation, and writing. They are progress targets: you should seek help promptly if you are falling substantially behind them.

Week of November 9: Question, scope, and proposal

Choose a topic, turn it into a focused question, and define a core project that can be completed even if an optional extension is not. By the end of the week, submit approximately one page containing:

1. the focused project question;
2. a short description of the modeled system;
3. state variables, parameters, and governing rules or equations;
4. the planned computational method;
5. expected plots, statistics, or other outputs;
6. at least one proposed verification test;
7. the minimum viable core model; and
8. one possible extension if time permits.

The proposal is a planning document, not an irreversible contract. Instructor feedback may reduce the scope or sharpen the question.

Week of November 16: Model and first implementation

Translate the project idea into a precise model and begin implementing it. By the end of the week, be prepared to show:

- the governing equations or update rules;
- a pseudocode, flowchart, or function-level implementation plan;
- one predictable test case or hand-worked update; and
- evidence that a meaningful piece of the implementation behaves as expected.

This is the time to discover whether the proposed model is computationally manageable. Reduce the scope now if the core calculation is not coming together.

November 23–24: Minimum viable simulation

Use the class days before the Thanksgiving break to connect the pieces. The core model should now run from start to finish. Be prepared to show:

- one complete baseline run;
- at least one diagnostic plot or quantitative summary;
- preliminary verification evidence; and
- a short list of known problems or uncertainties.

Do not wait for a polished result before reporting a problem. The purpose of this checkpoint is to identify difficulties while there is still time to diagnose them. The Thanksgiving break is not assumed to be a full project work week.

Week of November 30: Verification and investigation

Complete the verification tests and the controlled comparisons that address the project question. Interpret the results as they develop, and keep a record of failed approaches, numerical difficulties, and model limitations. By the end of the week, the substantive computation should be nearly complete so that the remaining time can be used for analysis and communication.

Final exam week: Synthesis and submission

Finish the analysis, figures, report or notebook, code documentation, and reproduction instructions. Check that every conclusion is supported by evidence and that another person could follow and run the work. Add an extension only if the verified core project and its interpretation are already complete.

Final deadline: Friday, December 11. The complete project must be submitted no later than December 11. This is the final, drop-dead submission date; plan to finish early enough to allow time for a last reproducibility check.

7 Final submission

Submit the following items:

1. **Project report or computational notebook.** It must include the question, model, assumptions, method, verification, investigation, results, difficulty record, limitations, and conclusions.
2. **Complete source code.** The submitted version must reproduce the reported results or clearly explain any external data or setup required.
3. **Reproduction instructions.** Include required software, the order in which files or notebook cells should be run, and any important run-time expectations.
4. **Sources and acknowledgments.** Cite data, equations, algorithms, documentation, code, and other material that informed the project.

Your final report need not recount every line of code. It should tell a coherent scientific and computational story: what you asked, what you built, how you know it works, what you learned, and where confidence in the result ends.

8 Evaluation rubric

Category	Weight	Evidence of strong work
Question, model, and assumptions	20%	Focused question; clear variables, parameters, equations or rules; assumptions and omissions identified.
Implementation and understanding	20%	Coherent implementation; important steps explained; libraries and adapted material used transparently; student demonstrates ownership rather than black-box use.
Verification and numerical reliability	20%	Relevant test with a stated expected result; evidence presented and interpreted; numerical limitations addressed where appropriate.
Investigation and interpretation	20%	Controlled comparisons answer the project question; results are analyzed rather than merely displayed; claims are supported by evidence.
Difficulties, limitations, and reflection	10%	A substantive difficulty is diagnosed thoughtfully; at least two limitations are connected to the strength of the conclusions.
Communication and reproducibility	10%	Clear organization and figures; complete code and instructions; sources acknowledged; another reader can follow and reproduce the work.

Project novelty, code length, visual complexity, and raw computational cost are not grading categories. A modest project can earn full credit through depth, correctness, careful investigation, and clear communication.

9 Final checklist

Before submitting, confirm that your project contains all of the following:

☐ A focused question that the simulation can address.

☐ A model description understandable without reading the code.

☐ Clearly stated variables, parameters, assumptions, and omissions.

☐ A complete, runnable core implementation.

☐ At least one concrete verification test.

☐ A controlled parameter, method, initial-condition, resolution, or stochastic comparison.

☐ A documented implementation difficulty and evidence-based diagnosis.

☐ Results connected explicitly to the original question.

☐ At least two limitations and their consequences for interpretation.

☐ Complete source code and reproduction instructions.

☐ Citations or acknowledgments for external ideas, data, and code.

Remember: The purpose of the project is not to prove that your first idea was correct or that your code worked immediately. The purpose is to show that you can formulate a model, make it computational, test it, learn from its failures, and use it responsibly to answer a question.